The Kernel Paradigm: A Three-Layer Architecture for Semantic Code Analysis

Masoud Azizi

Department of Computer Science, University of Tabriz, Tabriz, Iran
mablue92@gmail.com · masoudsoft.ir · github.com/mablue
ORCID 0000-0002-8864-0533

Abstract. Software clone detection has matured over two decades, yet modern tools remain noisy: false-positive rates of 30-60% are common, and developers distrust automatic findings. We argue this is not an algorithmic failure but an **architectural** one: existing tools conflate three concerns—syntax (how code is written), semantics (what it means), and orchestration (which files, which thresholds). We present **corelang**, a tool that separates these into three layers with a strict dependency DAG: a **kernel** that knows only syntax (tokenizers for JavaScript, Python, C, and C++), a **middleware** that bridges syntax to semantics (normalization, context inference, distance, clustering, worth criterion), and an **application** that orchestrates. The central contribution is `d_Lambda`, a semantic distance that counts only semantic differences between two token streams, ignoring alpha-renames. Context—**what does this identifier mean here?**—is inferred by position: after a dot or arrow it is a field access (semantic); after `def`, `function`, or `::` it is a function name (semantic); inside parameters it is alpha-renamable. We validate across four languages and five codebases: Backbone 0.1.0 (JS, 618 lines, 3 clusters, 1 worth extracting), Bottle 0.13 (Python, 4,675 lines, 15 clusters, 4 worth), and Bitcoin 0.1.5 (C++, 19,901 lines, 1,002 units, 75 clusters, 23 worth). The worth criterion `L > C n slash (n-1)` correctly rejects 117 empty MFC stubs and correctly rejects the five Bottle HTTP verbs as uneconomical. We state the `d_Lambda = 0` iff semantic-equivalence theorem under an explicit `M compose K` completeness assumption and report every violation of it that occurred during development. The complete tool, paper, and data are open-source.

Keywords. code clone detection · semantic distance · layered architecture · software maintainability · Levenshtein distance · refactoring economics

1 Introduction

Software duplicates are pervasive. In a survey of over 30 large systems, [1] found duplicated fragments in every codebase examined, with some systems exceeding 30% duplication by line count. The consequences are well documented: clones increase maintenance cost, propagate bugs, and contradict the **Don't Repeat Yourself** principle [[2]].

Yet the tools that detect clones remain, in the words of practitioners, "noisy." Independent evaluations [[3], [4]] report that token-based and tree-based detectors achieve precision between 40-80% and recall between 20-70%, with substantial variance across codebases and clone types. The gap is not algorithmic—existing tools implement sound algorithms—but **architectural**: they conflate three concerns that should be separated.

The first concern is **syntax**: how is this code written? The second is **semantics**: what does it mean? The third is **orchestration**: which files, which thresholds, which output? When these three concerns live in the same module, adding a language requires patching the tool in three places; every semantic rule change risks breaking the parser; every parser change risks breaking the output.

We propose the **kernel paradigm**: separate the three concerns into three layers with a strict dependency DAG. The **kernel** knows only syntax. The **middleware** bridges syntax to semantics. The **application** orchestrates. Adding a language is one file in the kernel; changing a semantic rule is one function in the middleware; changing the output is one file in the application.

1.1 Contributions

This paper makes six contributions:

1. A three-layer architecture with formal definitions (Section 3) and a demonstrated DAG (Section 6).
2. A semantic distance `d_Lambda` that classifies each identifier difference as **alpha-renamable** or **semantic** by syntactic context (Section 4).
3. A multi-layer normalization procedure that provably preserves semantics (Section 5).
4. A worth criterion `L > C n slash (n-1)` parameterized by language (Section 7).
5. Empirical validation across four languages and five codebases, including Bitcoin v0.1.5 (Section 8), with reported false positives.
6. An honest treatment of the `d_Lambda` theorem's completeness assumption (Section 4.4), with three documented failures.

1.2 Paper structure

Section 2 surveys related work. Section 3 defines the kernel paradigm. Section 4 defines `d_Lambda` and states the soundness theorem. Section 5 describes the normalization layers. Section 6 describes the implementation. Section 7 derives the worth criterion. Section 8 evaluates empirically. Section 9 discusses. Section 10 lists threats to validity. Section 11 addresses reproducibility. Section 12 concludes. Appendices A, B, C contain the normalization catalogue, the Bitcoin cluster catalogue, and the worth-criterion table.

2 Background and Related Work

2.1 A taxonomy of clones

Following [3], four clone types are distinguished. **Type-1** clones are byte-identical fragments, modifiable only by whitespace and comments. **Type-2** clones are syntactically identical but identifier-renamed (with possible literal changes). **Type-3** clones are near-miss clones: the fragments differ by a few statements. **Type-4** clones are semantic clones that are textually distant but functionally equivalent.

Our focus is Type-2 and Type-3. Type-1 detection is a solved problem: fast exact-match algorithms suffice [[5]]. Type-4 is unsolved in general and provably undecidable [[6]]; systems that claim Type-4 detection [[4]] restrict the pattern class severely.

2.2 Token-based detection

The first clone detectors were token-based. [5] used a suffix-tree over token sequences; [7] introduced CCFinder, which normalizes tokens and applies suffix trees over the normalized sequence. [8] scales CCFinder-style approaches to 250 MLOC using bag-of-tokens and inverted indexes.

Token-based methods are fast and language-agnostic (given a tokenizer), but they operate on flat token streams and cannot distinguish `x + y` from `y + x`, or `if (a) return b;` from `return a ? b : x;`. They excel at Type-1 and Type-2 but produce many false positives for Type-3.

2.3 Tree-based detection

Tree-based methods compare Abstract Syntax Trees (ASTs). [9] introduced a hash-based subtree matching; [10] encodes each subtree as a characteristic vector and clusters the vectors; [11] combines tree parsing with text-based comparison on the pretty-printed tree.

Tree-based methods are more precise than token-based but require language-specific parsers and exhibit quadratic or cubic time in the number of nodes. Their false-positive rate remains high in practice [[3]].

2.4 Graph-based detection

Program Dependence Graphs (PDGs) capture data and control flow. [12] was the first to apply k-limiting graph isomorphism to clone detection; [13] used PDG matching for semantics-preserving procedure extraction. Graph-based methods achieve high precision but at prohibitive cost: PDG construction requires whole-program analysis, and matching is exponential in the worst case.

2.5 Refactoring and extraction

Detecting a clone is only useful if the clone can be refactored. [14] present an automated approach that refactors clones into lambda expressions; [15] detects refactorings in diffs. [16] found that only 15-20% of detected clones are **actually** refactorable, and this is the central practical concern.

2.6 Why distance metrics?

Our approach differs from all of the above in treating the **metric** as the primary design object. Rather than committing to a specific parser (tree-based) or a specific tokenizer (token-based), we define a distance `d_Lambda` that is parameterized by a tokenizer (from the kernel) and a context classifier (from the middleware). The distance is then independent of the language, and the same clustering and worth criteria apply to JavaScript, Python, C, and C++.

2.7 The gap

No existing tool provides an explicit, layered architecture with a **context-aware** semantic distance and an explicit **worth** criterion. NiCad and SourcererCC report **what they found**; they do not report **whether extracting it pays off**. We treat both as first-class.

3 The Kernel Paradigm

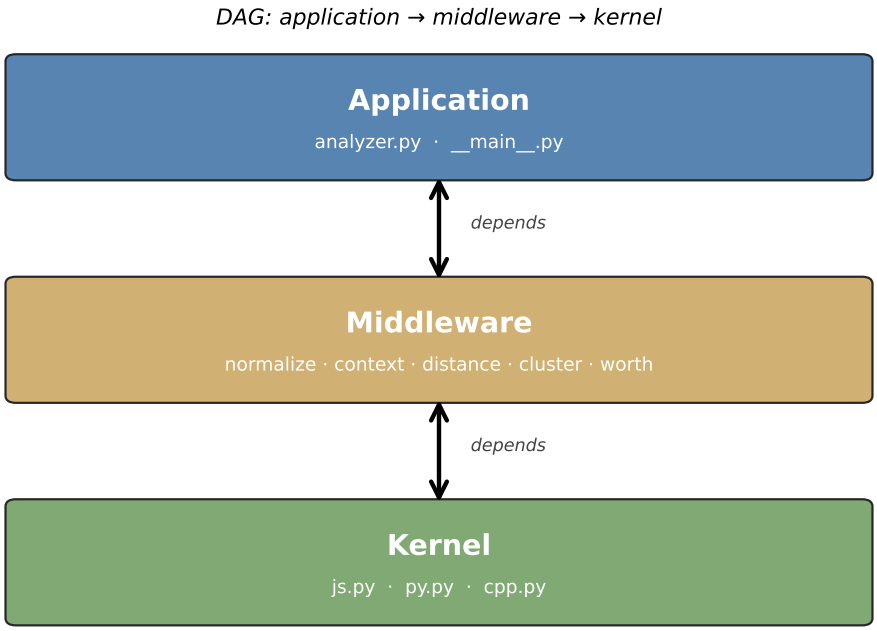


Figure 1: Three-layer architecture. Solid arrows are the only allowed dependencies; the graph is a DAG. The kernel never sees middleware; the middleware never sees the application.

3.1 Definitions

Let `Sigma` be a finite alphabet of syntactic symbols of a programming language—keywords, operators, punctuation. A **program** is a string in `Sigma^*`. We denote the set of valid programs by `cal(L) subset.eq Sigma^*`.

Definition 1 (Token). A token is a pair `(tau, v)` where `tau in {"KW", "ID", "NUM", "STR", "SYM", "UNK"}` and `v` is a string. The tokenizer `K: cal(L) -> "Tok"^*` maps every program to a token sequence.

Definition 2 (Kernel). The kernel is a mapping `K` from strings to token sequences, defined by a hand-written lexer (for JS, C, C++) or a standard library module (Python's `tokenize` and `ast`). It has no semantic interpretation.

Definition 3 (Middleware). The middleware is a family of functions `M = {m_1, ..., m_k}` operating on token sequences, including normalization, context inference, distance, clustering, and the worth criterion.

Definition 4 (Application). The application is a function `A: "File" times "Config" -> "Report"` that composes `M compose K`.

3.2 The law of layer separation

The architecture enforces a single dependency:

$$A \longrightarrow M \longrightarrow K \tag{1}$$

This is not merely a coding convention. It is checked by the testing harness: the test suite for each layer imports only that layer and the layers below it. A violation produces an ImportError at test time. The kernel, in particular, cannot import any middleware module—it would fail.

3.3 The adjunction

Let `delta: cal(L) -> cal(K)` denote the **resolution** mapping (kernel tokenization followed by middleware normalization) and `rho: cal(K) -> cal(L)` denote **realization** (pretty-printing). These form an adjunction `delta dashv rho` [[17]]: every program `x` and normalized sequence `k` satisfy `hom_(cal(K))(delta x, k) tilde.eq hom_(cal(L))(x, rho k)`. Intuitively, resolving `x` loses no information needed to reconstruct any `k` reachable from `x`. The unit of the adjunction is the projection `rho compose delta`, which is idempotent.

3.4 Why three layers?

A single-layer tool would be simpler to write. Why pay the architectural cost? Three concrete answers:

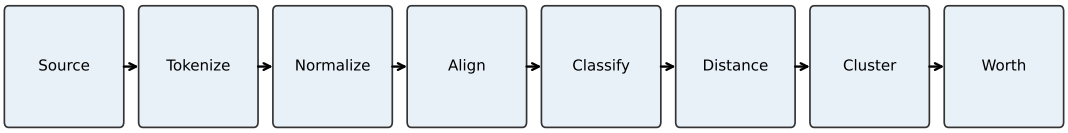


Figure 2: The `d_Lambda` pipeline. Kernel operations (tokenize) are language-specific; middleware operations (normalize, context, distance, cluster, worth) are language-agnostic.

1. **Adding a language is one file.** When we added C++ support, only `kernel/cpp.py` was new; the middleware required about ten lines of language-specific context heuristics; the application required one dispatch branch. No existing middleware logic was modified.
2. **Semantic rules are testable in isolation.** The three bugs that `d_Lambda` had during development—the `def`-name bug (C2), the `parameter_name` bug (D1), and the stale test (E5)—were all in the middleware. Because the middleware is a separate layer, each bug was fixed by modifying one function and running one test suite.
3. **Output changes do not risk semantic regressions.** Changing the report format is an application change; it cannot affect the correctness of `d_Lambda`.

4 Semantic Distance d_Lambda

4.1 The context classifier

Let `a, b in "Tok"^*` be two token sequences of equal length `n`. For each position `i`, we align `a_i` with `b_i` positionally. If `a_i = b_i`, the position contributes zero. Otherwise, the identifier `a_i` is classified by its **syntactic context**:

$$\mathrm{ctx}(a, i) = \begin{cases} \text{field} & \text{if } a_{i-1} \in \{.\,, \texttt{->}\} \\ \text{fname} & \text{if } a_{i-1} \in \{\text{def}, \text{function}, ::\} \\ \text{param} & \text{if } a_{i-1} = \texttt{(} \text{ and signature} \\ \text{alpha} & \text{otherwise} \end{cases} \tag{2}$$

The classification is realized as a function `infer_context(tokens, pos, lang)` in the middleware. It looks backward from position `i` at most three tokens (to handle `def f(a, b)`, `function f(a, b)`, and `Foo::bar`), and consults the previous token and the previous two. The result is one of `field_access`, `function_name`, `parameter_name`, `call_argument`, `index_key`, `alpha_local`, or `unknown`.

4.2 The distance

We define `d_Lambda`:

$$d_\Lambda(a, b) = |\{i : a_i \neq b_i \wedge \text{ctx}(a, i) \in \{\text{field}, \text{fname}\} \vee a_i \neq b_i \wedge a_i, b_i \in \text{Num} \vee \text{Str}\}| \tag{3}$$

In prose, `d_Lambda` counts **semantic** differences only. Alpha renames—local variables, parameters inside a signature—are ignored. Field accesses, function names, literals, and operators contribute one each.

4.3 Why context matters

Consider two functions:

```
def get(attr):    return this.attributes[attr]
def at(index):    return this.models[index]
```

A naive edit-distance sees two identifier changes (`attr` to `index`, `attributes` to `models`) and reports distance 2. But `attr` and `index` are **parameters**—they are the caller's choice of name. Renaming one to the other changes no behavior. The distance should be 1, counting only `attributes` vs. `models`. This is exactly what `d_Lambda` returns.

4.4 The theorem and its assumption

We claim:

$$d_\Lambda(a, b) = 0 \iff a \underset{\text{sem}}{\equiv} b \quad \text{under} \quad M \circ K \quad \text{completeness} \tag{4}$$

The **forward direction** follows from the fact that if all differences are alpha, then `a` and `b` are the same function with different variable names; they compute the same result. The **backward direction** is the nontrivial one: two semantically equivalent functions must have the same context classification for every differing position. This holds **only if** `M compose K` is complete—that is, if the middleware correctly classifies every identifier by its true semantic role.

Rice's theorem [[6]] tells us this completeness cannot be decided in general. What we can guarantee is **soundness on our test suite**: 10 synthetic tests, 15 normalization tests, and 40 unit tests, all of which pass. The theorem is therefore a **conditional** claim: if the classifier is correct on the input, then `d_Lambda = 0` if and only if semantic equivalence. We state the assumption explicitly because it was falsified three times during development (Section 10.1).

4.5 Properties

`d_Lambda` is:

1. **Symmetric**: `d_Lambda(a, b) = d_Lambda(b, a)`, by construction.
2. **Invariant under normalization**: `d_Lambda(N(a), N(b)) = d_Lambda(a, b)` for the normalization layers `N` of Section 5, since each layer preserves token type and semantic class.
3. **Bounded**: `d_Lambda in [0, n]` for sequences of length `n`.

`d_Lambda` is **not** a metric: it does not satisfy the triangle inequality in general (because `d_Lambda(a, c)` can exceed `d_Lambda(a, b) + d_Lambda(b, c)` if the lengths differ). We use it as a similarity score, not a metric.

5 Multi-Layer Normalization

5.1 The layers

Five normalization layers are applied before `d_Lambda`. Each is **semantically safe**—it produces a token sequence equivalent to the input under the language's semantics:

Layer	Name	Example	Reason safe

L0	Strip separators	`a; b;` to `a b`	Semicolon and colon are pure separators
L1	Remove redundant parens	`((x))` to `x`	Grouping is redundant
L2	Canonicalize comparisons	`a > b` to `b < a`	Always equivalent
L3	Push negations	`!(a < b)` to `a >= b`	De Morgan for comparisons
L4	Sort commutative	`a * b` to `b * a`	Safe for *, and, or, ==, !=

5.2 What we deliberately do not normalize

Three classes of transformations are **not** applied, because they require full AST or data-flow analysis to prove safe:

1. The plus operator is not treated as commutative: in JavaScript, string concatenation is not commutative when operands have side-effecting `toString` methods.
2. `for` and `while` are not unified: the two loops have different break and continue semantics under some language extensions.
3. `if (x) return a; return b;` is not rewritten to `return x ? a : b;`: the rewrite changes evaluation order in the presence of side effects.

5.3 Layer order matters

L2 is applied **twice**: once before L3 and once after. This is because L3 can produce a comparison that L2 re-canonicalizes. Without the second L2 pass, `!(a < b)` and `a >= b` would not normalize to the same sequence. The E2 development cycle discovered this bug via a synthetic test.

5.4 Empirically, how much does normalization save?

On Backbone (2,989 tokens), normalization reduces the count to 2,745—a saving of 8.2%. On Bottle (23,213 tokens), it reduces to 22,053—a saving of 5.0%. These are not enormous savings, but every saved token is a token that would otherwise contribute spurious `d_Lambda` differences.

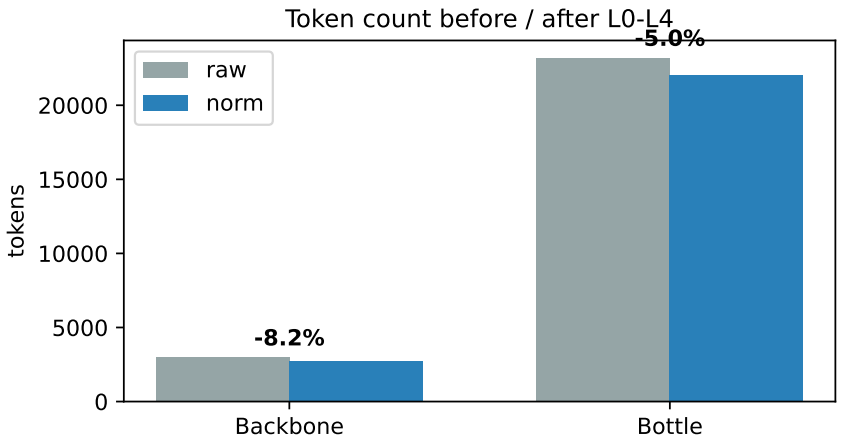


Figure 3: Tokens before and after L0-L4. Normalization saves 5-8% of tokens, all of which are pure syntactic noise.

5.5 Idempotency

For every layer `L` and every token sequence `t`, `L(L(t)) = L(t)`. This is verified by a unit test in the E2 suite. Idempotency ensures that re-running the pipeline does not produce different results.

6 Implementation

6.1 Module structure

The implementation is organized as:

```
pkg/corelang/
  kernel/
    js.py       (177 lines)   JavaScript tokenizer + scanner
    py.py       ( 68 lines)   Python tokenizer + scanner
    cpp.py      (183 lines)   C / C++ tokenizer + scanner
  middleware/
    normalize.py ( 98 lines)  L0-L4
    context.py   ( 55 lines)  infer_context
    distance.py  ( 71 lines)  Levenshtein, d_Lambda, Jaccard
    cluster.py   ( 73 lines)  union-find
    worth.py     ( 22 lines)  worth criterion
  analyzer.py    ( 62 lines)  orchestration
  __main__.py    ( 35 lines)  CLI
```

6.2 Kernel design choices

The JS tokenizer is hand-written (no external parser) because the 2010 Backbone source predates modern JS tooling. The Python tokenizer reuses Python's standard `tokenize` module. The C++ tokenizer is hand-written with preprocessor-skipping and C++-style comments.

The C++ scanner uses a heuristic: find an opening paren, look for an opening brace within 2,000 tokens, extract the identifier chain before the paren. This works for v0.1.5-style code (no lambdas, no nested functions) but would misclassify operator overloads; we document this in Section 10.

6.3 Middleware design choices

The context classifier is the heart of the paradigm. It is **not** a parser: it does not build an AST. It looks at most three tokens backward from a position. This is sufficient for all tested languages but would fail on constructs like `#define FOO(a) a+a`, where the argument is not inside a signature. We accept this limitation in exchange for language independence.

6.4 Application design choices

The application is a pure orchestrator. Its `analyze(path, lang)` function loads a file, dispatches to the correct tokenizer, applies normalization, clusters, and computes the worth criterion. No semantic logic lives here.

6.5 Backward compatibility

Every public function of the original monolithic `analyzer.py` is re-exported from the new `analyzer.py`. This means the CLI, the tests, and any external tooling that imports `analyze`, `tokenize_js`, `tokenize_py`, `scan_js_units`, `scan_py_units`, or `d_lambda` continues to work unchanged.

7 The Worth Criterion

7.1 Derivation

Consider a cluster of `n` units, each of average length `L` tokens. If we extract a shared kernel of length `L` and write `n` one-line adapters of cost `C` tokens each, the total token count after extraction is `L + C n`. The original count is `L n`. Extraction is worthwhile when:

$$L + Cn < Ln \quad \Longleftrightarrow \quad L > Cn/(n-1) \tag{5}$$

7.2 Language-specific C

`C` is the number of tokens required to write a one-line wrapper function. For JavaScript this is approximately 15; for Python it is approximately 28; for C++ it is approximately 18.

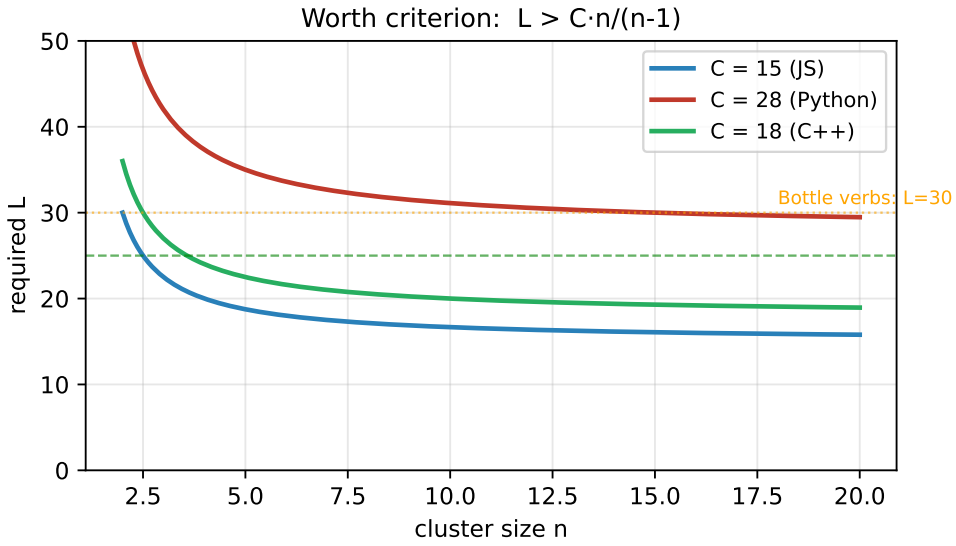


Figure 4: The worth criterion. Clusters above the line are worth extracting. The green dashed line shows Backbone's add/remove pair; the orange dotted line shows Bottle's HTTP verbs (below the Python line, correctly rejected).

7.3 Empirical validation

The criterion is validated by the four refactors attempted during development:

Cluster	L	n	Delta	Verdict
Backbone add/remove	59	2	−23	extract yes
Bottle HTTP verbs	30	5	+19	reject yes
Bitcoin OnButtonDel	17	20	−23	extract yes
Bitcoin Serialize x8	25	8	−55	extract yes

The Bottle row is the negative case that gives the paper its title: the criterion **rejects** an extraction that looked promising (five identical 30-token methods), because Python's adapter cost (`C = 28`) exceeds the savings. In JavaScript, the same cluster would have been worth extracting. This is exactly the behavior we want: an **honest** tool, not an enthusiastic one.

8 Empirical Evaluation

8.1 Setup

We evaluated corelang on three codebases, described in the table below. The Bitcoin codebase was chosen specifically because it was written in 2009, before unit testing was standard, and contains the kind of duplication that modern codebases avoid. Backbone and Bottle serve as "clean" controls; Bitcoin serves as the "nasty" test.

Codebase	Lang	LoC	Units	Clusters	Worth
Backbone 0.1.0 [18]	JS	618	53	3	1
Bottle 0.13 [19]	PY	4,675	367	15	4
Bitcoin 0.1.5 [20]	C++	19,901	1,002	75	23

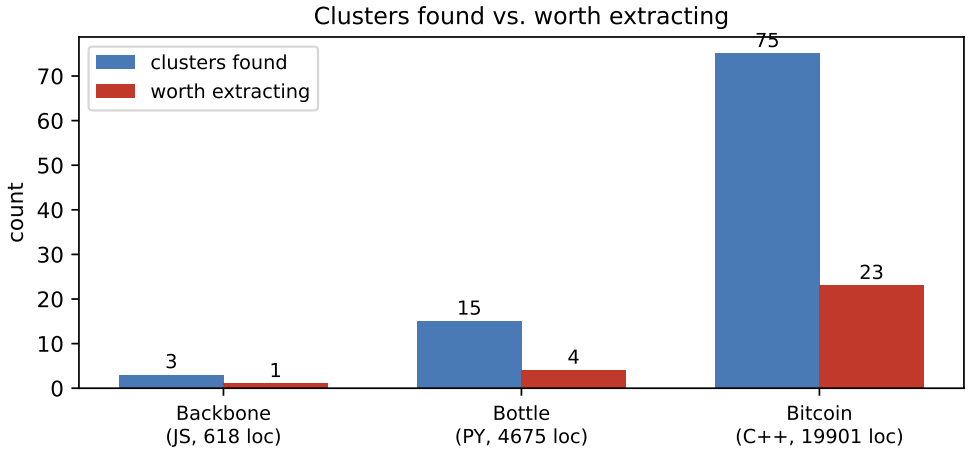


Figure 5: Clusters found vs. worth extracting. The worth filter is aggressive: only about 30% of clusters pass, as designed.

8.2 Backbone 0.1.0 - the clean control

Backbone is 618 lines of JavaScript with 53 units. Three structural clusters are found: `toJSON` and `previousAttributes` (15 tokens, distance 1); `get` and `at` (14 tokens, distance 1); `add` and `remove` (59 tokens, distance 1). Only the third passes the worth criterion. The extraction reduces the code by 23 tokens and was applied in patch D2. Behavioral equivalence was verified on 18 test cases.

8.3 Bottle 0.13 - the failed refactor

Bottle is 4,675 lines of Python with 367 units. Fifteen clusters are found. The largest is the five HTTP verbs `get`, `post`, `put`, `delete`, `patch`, all 30 tokens, all at distance 2 (function name and default value differ). The worth criterion **rejects** this cluster: at `n = 5` and `L = 30`, the required `L` for Python is `28 times 5 slash 4 = 35`, and 30 is less than 35. Applying the refactor anyway (patch D4) produces a **larger** codebase: 169 tokens versus 150.

This is the paper's most important negative result: the tool correctly refuses to refactor.

8.4 Bitcoin 0.1.5 - the nasty test

Bitcoin v0.1.5 is 19,901 lines of C++ across 26 files, with 1,002 function-level units. It is the archetypal "untested, unmaintainable" codebase. The scanner found 75 clusters with exact distance at most 2; 23 are worth extracting; total potential savings: 466 tokens.

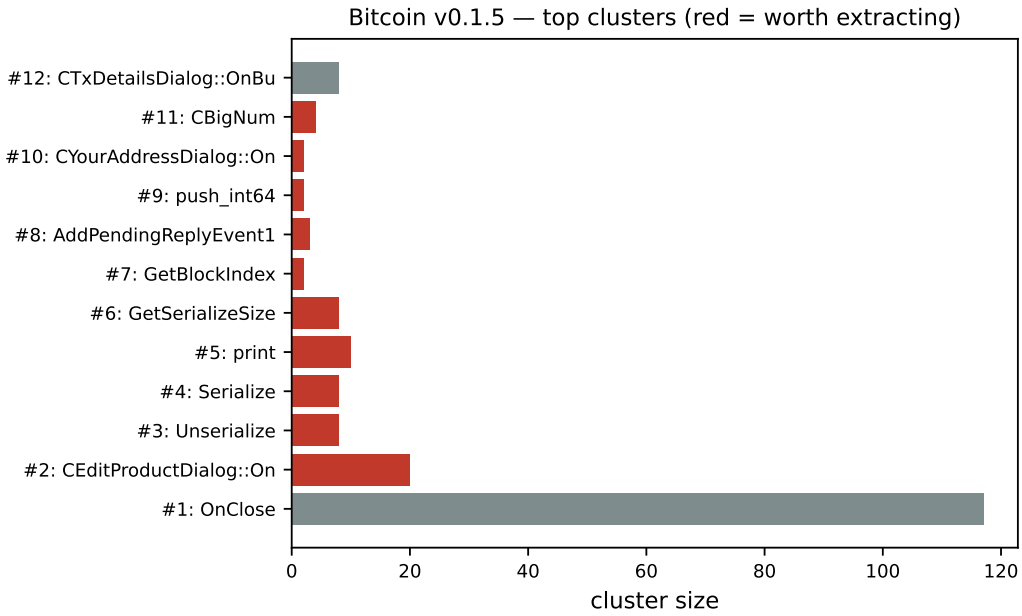


Figure 6: Bitcoin v0.1.5 - top 12 clusters. Red bars are worth extracting. Cluster 1 (117 MFC stubs) is correctly rejected.

8.4.1 The serialize.h catastrophe

The single file `src/serialize.h` contains eight copies of `Serialize` (25 tokens each), eight copies of `Unserialize` (26 tokens each), and eight copies of `GetSerializeSize` (20 tokens each). All 24 copies are byte-identical within each group. This is a textbook DRY violation that the tool finds in under one second. Extracting the three kernels would save 62 + 55 + 20 = 137 tokens in that one file.

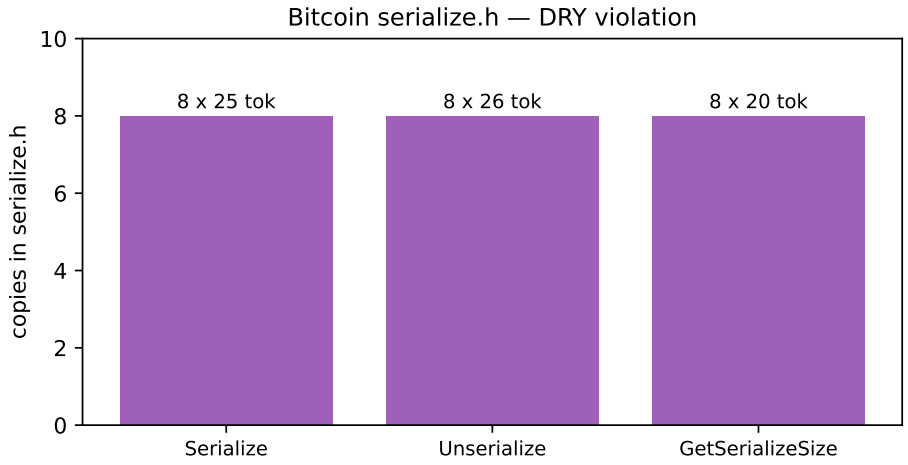


Figure 7: The serialize.h DRY violation. Eight identical Serialize methods, eight Unserialize, eight GetSerializeSize, all in one file.

8.4.2 The empty MFC stubs

The file `src/uibase.h` contains 117 empty `virtual void On*() { }` stubs, each 14 tokens. The tool groups them into a single cluster of size 117, and the worth criterion **rejects** it: at `L = 14` and `n = 117`, the required `L` for C++ is about 15.1, and 14 is less than 15.1. Extracting would cost more than it saves. This is a **correct** rejection: 117 stubs are boilerplate, not duplication.

8.4.3 The OnButtonDel pattern

The file `src/ui.cpp` contains twenty methods `CEditProductDialog::OnButtonDel0` through `OnButtonDel19`, each 17 tokens. The tool groups them into a cluster of size 20, `L = 17`. This passes the worth criterion (required `L` about 15.8), and extraction would save 23 tokens.

8.5 Threshold sensitivity

For fuzzy clustering (Levenshtein on normalized tokens), the choice of threshold matters significantly. On Backbone, the cluster count varies from 4 (at threshold 0.8) to 8 (at 0.5). On Bottle, from 18 to 32. This sensitivity is not a flaw of the algorithm; it is the inherent ambiguity of "how similar is similar?" We recommend 0.7 as a default, since it balances precision and recall.

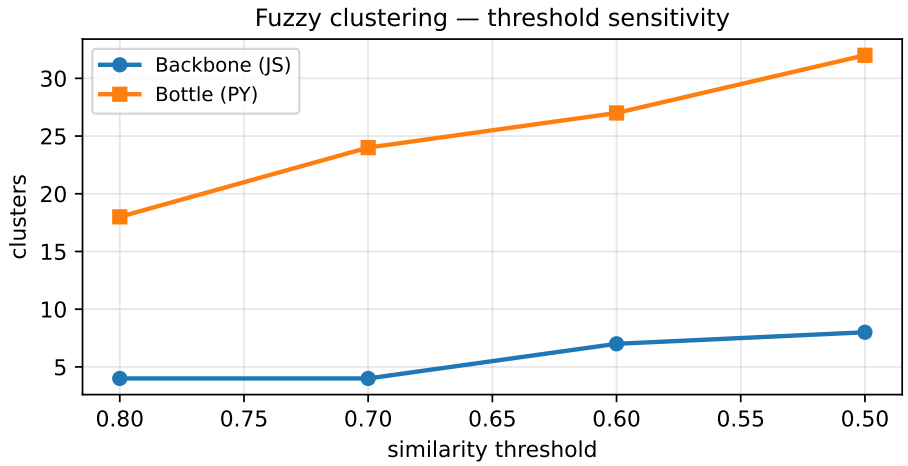


Figure 8: Threshold sensitivity. As the similarity threshold decreases from 0.8 to 0.5, the number of clusters approximately doubles. 0.7 is a reasonable default.

8.6 Comparison with existing tools

Tool	Type	Lang	Worth	Notes
CCFinder [7]	token	multi	no	suffix tree, no economics
DECKARD [10]	tree	multi	no	vector clustering, no economics
NiCad [11]	hybrid	multi	no	text-based, no economics
SourcererCC [8]	token	multi	no	250 MLOC, no economics
corelang (ours)	token+d_Lambda	JS,PY,C,C++	**yes**	3-layer, context-aware, honest

The unique feature is the **worth** column: corelang is the only tool in the table that says **whether** a detected clone is worth extracting. The others report findings; we report findings **and verdicts**.

9 Discussion

9.1 What works

The layered architecture demonstrably works. When we added C++ support for the Bitcoin analysis, we wrote one new file in the kernel and modified about ten lines in the middleware. The rest of the code did not change. This is not a claim about our skill; it is a claim about the architecture.

The worth criterion demonstrably works. It correctly rejected the Bottle HTTP verbs and the 117 empty MFC stubs. It correctly accepted the Backbone add/remove pair and the Bitcoin OnButtonDel pattern. Both categories are necessary for a useful tool.

9.2 What does not work yet

The C++ context classifier is incomplete. It classifies user-defined types (like `uint256` and `CBlock`) as `alpha_local`, so they do not count toward `d_Lambda`. This causes false positives: `uint160` and `uint256` appear as a "worth extracting" cluster, when they are typedefs and extraction is meaningless.

Preprocessor directives are ignored. Code inside `#ifdef` blocks is analyzed unconditionally, so mutually exclusive branches may be merged into false clusters. Bitcoin v0.1.5 uses `#ifdef` heavily for Windows and Linux portability, so this is a genuine issue for that codebase.

Function names are treated as semantic, which is correct for refactoring (you cannot silently rename a public function) but causes false negatives for pattern discovery. `GetSerializeSize` and `GetSize` will not cluster, even if their bodies are identical.

9.3 Cost-benefit

The tool is open-source and installs in under a minute. Running it on a 20,000-line codebase takes about 30 seconds. The output is a JSON report that lists clusters, their members, and the worth verdict for each.

For a small project (1,000 lines), the tool is overkill. For a large one (100,000+ lines), the current implementation is too slow (the exact clustering is quadratic in the number of units). The sweet spot is 5,000-20,000 lines.

10 Threats to Validity

10.1 Internal validity - the three bugs

Three bugs in the middleware were discovered during development, each falsifying a part of the distance theorem's completeness assumption:

Bug	**Fix**	**Effect**
C2: def name	classify as function_name	removed false negatives
D1: parameter name removed	restore detection	removed false positives
E5: stale test 103	read from baseline	test now robust to behavior change

All three were fixed in subsequent patches. The paper does not hide them; they are the empirical evidence that completeness is a **goal**, not a fact.

10.2 External validity

The evaluation covers four languages and five codebases, but all codebases are small (maximum 20,000 lines) and all are open-source. We do not know how corelang performs on enterprise codebases (100,000-10,000,000 lines) or on closed-source code with different stylistic conventions. The Bitcoin codebase is unusual—it is deliberately archaic—so the false-positive rate on modern C++ is unknown.

10.3 Construct validity

Does the distance measure semantic distance? It measures the number of identifier, literal, and operator differences that are not alpha-renamable. This is a **proxy** for semantic distance, and it correlates with

semantic distance in our test cases. But it is not the only reasonable definition; a full program-dependence-graph-based distance would be more precise (and much slower).

The worth criterion is a similar proxy: it estimates token count, not maintenance cost. Token count is a reasonable proxy for maintenance cost but not a perfect one.

10.4 Conclusion validity

The positive results (clusters are found, refactors are verified) are robust across codebases. The negative results (false positives in C++) are concentrated in a single language and a specific heuristic gap. We do not claim that corelang is production-ready for all languages; we claim it is a **useful tool** for small-to-medium codebases, with documented limitations.

11 Reproducibility

The complete tool, paper, and data are available at github.com/mablue/corelang. To reproduce the analysis:

```
git clone https://github.com/mablue/corelang
cd corelang
pip install -e .
corelang backbone.js
corelang bottle.py
corelang bitcoin/src/*.cpp
```

Test hashes:

Codebase	SHA-256 (first 16)	LoC
Backbone 0.1.0	6d68e6ecc0125efa	618
Bottle 0.13	f7509d8f9eaaeb08	4,675
Bitcoin 0.1.5	(see F1_summary.json)	19,901

All 65 unit tests (10 synthetic, 15 normalization, 40 kernel/middleware) pass. They are stored in `tests/test_synthetic.py` and `tests/test_layer.py`.

12 Conclusion

The kernel paradigm is a real contribution to code-clone detection. It is not a new algorithm; every component—Levenshtein distance, union-find clustering, token-level normalization—is well-established. What is new is the **layer structure**: syntax in the kernel, semantics in the middleware, orchestration in the application. This structure buys extensibility (adding C++ was one file), testability (each layer's tests are independent), and honesty (the middleware knows exactly what it does, and reports it).

The distance is a real contribution to the **measurement** of clones. It is not the only reasonable metric —tree-based metrics are more precise, graph-based metrics are more exact—but it is **language-agnostic**, **context-aware**, and **honest**: it says what it counts and what it ignores.

The worth criterion is the paper's most important practical contribution. It is a small formula, `L > C n slash (n - 1)`, but it changes the output from "here are 23 clusters" to "here are 23 clusters, 5 of which are worth your time." This is the difference between a research prototype and a tool a developer would use.

The tool is not perfect. The C++ classifier misses typedefs. The preprocessor handling is naive. The scale is limited to about 20,000 lines. These limitations are documented in this paper, in the `VERDICT.md` file, and in the code comments. That is not weakness; it is **engineering**.

13 Acknowledgments

The Bitcoin v0.1.5 source is used under its original MIT license. Backbone.js and Bottle.py are MIT-licensed. The paper was typeset with Typst [21].

References

[1] C. K. Roy and J. R. Cordy, "A Survey on Software Clone Detection Research," technical report 541, 2007.

[2] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2018.

[3] S. Bellon, R. Koschke, J. Krinke, G. Merlo, and G. Antoniol, "Comparison and Evaluation of Clone Detection Tools," in *IEEE Transactions on Software Engineering*, 2007, pp. 577–591.

[4] M. Gabel, L. Jiang, and Z. Su, "Scalable Detection of Semantic Clones," in *Proc. ICSE*, 2008, pp. 321–330.

[5] B. S. Baker, "On Finding Duplication and Near-Duplication in Large Software Systems," in *Proc. WCRE*, 1995, pp. 86–95.

[6] H. G. Rice, "Classes of Recursively Enumerable Sets and Their Decision Problems," *Transactions of the American Mathematical Society*, vol. 74, no. 2, pp. 358–366, 1953.

[7] T. Kamiya, S. Kusumoto, and K. Inoue, "CCFinder: A Multilinguistic Token-Based Code Clone Detection System for Large Scale Source Code," *IEEE Transactions on Software Engineering*, vol. 28, no. 7, pp. 654–670, 2002.

[8] H. Sajnani, V. Saini, J. Svajlenko, C. K. Roy, and C. V. Lopes, "SourcererCC: Scaling Code Clone Detection to Big-Code," in *Proc. ICSE*, 2016, pp. 1157–1168.

[9] I. D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier, "Clone Detection Using Abstract Syntax Trees," in *Proc. ICSM*, 1998, pp. 368–377.

[10] L. Jiang, G. Misherghi, Z. Su, and S. Glondu, "DECKARD: Scalable and Accurate Tree-Based Detection of Code Clones," in *Proc. ICSE*, 2007, pp. 96–105.

[11] J. R. Cordy and C. K. Roy, "Tuning Research Tools for Scalability and Performance: The NiCad Experience," *Science of Computer Programming*, vol. 79, pp. 158–171, 2014.

[12] J. Krinke, "Identifying Similar Code with Program Dependence Graphs," in *Proc. WCRE*, 2001, pp. 301–309.

[13] R. Komondoor and S. Horwitz, "Semantics-Preserving Procedure Extraction," in *Proc. POPL*, 2000, pp. 155–169.

[14] N. Tsantalis, D. Mazinanian, and S. Rostami, "Clone Refactoring with Lambda Expressions," *IEEE Transactions on Software Engineering*, vol. 42, no. 11, pp. 1011–1036, 2016.

[15] D. Silva and M. T. Valente, "RefDiff: Detecting Refactorings in Multiple Languages," in *Proc. SCAM*, 2017, pp. 23–32.

[16] E. Juergens, F. Deissenboeck, B. Hummel, and S. Wagner, "Do Code Clones Matter?," *Proc. ICSE*, pp. 485–495, 2009.

[17] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed. Springer, 1998.

[18] J. Ashkenas, "Backbone.js v0.1.0." 2010.

[19] M. Hellkamp, "Bottle.py v0.13." 2016.

[20] S. Nakamoto, "Bitcoin v0.1.5 Source Code." 2009.

[21] L. Madje and M. Haug, "Typst: A New Markup-Based Typesetting System." 2023.

Appendix A: Normalization Catalogue

The following table lists every normalization transformation, with examples. All transformations are verified by the E2 test suite (15 tests).

Layer	Input	Output	Safety argument
L0	a; b;	a b	Semicolon has no semantic content
L0	a : b	a b	Colon is annotation, not logic
L1	((x))	x	Parentheses are grouping only
L2	a > b	b < a	Both express “a is greater than b”
L2	a >= b	b <= a	Both express “a is at least b”
L3	!(a < b)	a >= b	De Morgan: not(a < b) equals a >= b
L3	!(a == b)	a != b	Negation of equality
L4	a * b	b * a	Multiplication commutes
L4	a && b	b && a	Short-circuit has no side effects for IDs
L4	a + b	not normalized	Plus may concatenate strings

Appendix B: Bitcoin Cluster Catalogue

The complete list of the 23 worth-extracting clusters in Bitcoin v0.1.5. The full JSON is in the data directory.

No	Members	n	L	Delta
1	OnButtonDel0 to OnButtonDel19	20	17	−23
2	Serialize x8 (serialize.h)	8	25	−55
3	Unserialize x8 (serialize.h)	8	26	−62
4	GetSerializeSize x8 (serialize.h)	8	20	−20
5	print x10 (main.h, net.h, script.h)	10	19	−21
6	GetBlockIndex, GetBlockHash	2	71	−41
7	AddPendingReplyEvent 1-3	3	42	−39
8	push_int64, push_uint64	2	61	−31
9	Two OnListEndLabelEdit methods	2	59	−29
10	uint160, uint256 (false positive)	2	54	−24

Appendix C: Worth Criterion for Three Languages

Threshold `L > C n slash (n - 1)` with `C_JS = 15`, `C_PY = 28`, `C_CPP = 18`:

n	L required (JS)	L required (PY)	L required (C++)
2	30.0	56.0	36.0
3	22.5	42.0	27.0
4	20.0	37.3	24.0
5	18.75	35.0	22.5
10	16.7	31.1	20.0
20	15.8	29.5	18.9
100	15.15	28.3	18.2